

BIOLOGICAL DATA ANALYSIS WITH R

R Objects: Vectors, Matrices & Arrays

Hands-on session · creation · operations · indexing

QUICK RECAP

Where We Left Off

Last session covered the basics of R as a language and environment.

1

R Environment

Console, script editor, environment & plots panes

2

Variables & Assignment

Using `<-` to create and store values

3

Data Types

numeric, character, logical, integer

4

Basic Operators

Arithmetic, relational and logical operators

Today, we move from single values to collections of values — the data structures R is built on.

Learning Objectives

1

Create & manipulate vectors

Build vectors, understand vector types, apply vectorised operations

2

Index & subset data

Access, filter, and modify elements using positional and logical indexing

3

Build matrices & arrays

Construct 2-D and n-D structures and perform matrix arithmetic

4

Apply skills hands-on

Three guided exercises to practice each concept in RStudio

```
today.R

# Vectors
v <- c(1,2,3)
v[2]

# Matrices
m <- matrix(1:6,
            nrow=2)

# Arrays
a <- array(1:24,
           dim=c(2,3,4))
```

The Building Blocks of Data in R

R organises data into structures of increasing dimensionality. We build on vectors to reach matrices and arrays.

1-D

Vector

Single data type

TODAY

2-D

Matrix

Rows & columns, one type

TODAY

n-D

Array

Multiple dimensions, one type

TODAY

1-D

List

Mixed data types (next session)

2-D

Data Frame

Table of mixed types (next session)

Creating Vectors

A vector is an ordered collection of values of the same type — R's most fundamental data structure.

c()

Combine values into a vector

`c(1, 2, 3, 4)`**:**

Generate a sequence

`1:10`**seq()**

Sequence with custom step

`seq(0, 20, by = 5)`**rep()**

Repeat values

`rep("A", times = 3)`

```
vectors.R

> nums <- c(1, 2, 3, 4, 5)
> nums
[1] 1 2 3 4 5

> chars <- c("a", "b", "c")

> seqn <- seq(0, 20, by=5)
> seqn
[1] 0 5 10 15 20

> rep("A", times = 3)
[1] "A" "A" "A"
```

Vector Types & Operations

Atomic Vector Types

numeric 3.14, 100, -5

character "gene", "cell"

logical TRUE, FALSE

integer 1L, 2L, 3L

Vectorised Arithmetic

Operations apply element-by-element — no explicit loop needed. $+$ $-$ $\times$ $/$ $^$ work directly on whole vectors.

```
vector_ops.R

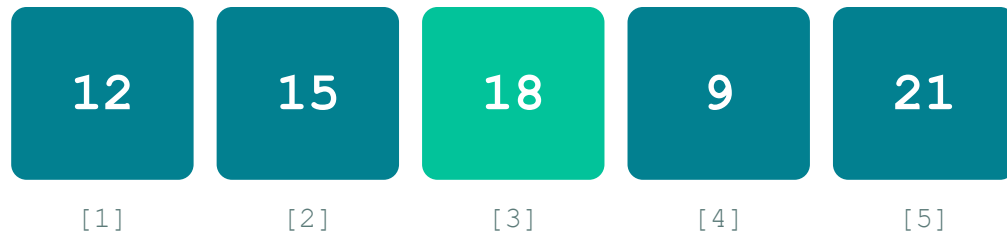
> a <- c(10, 20, 30)
> b <- c(1, 2, 3)

> a + b
[1] 11 22 33
> a * b
[1] 10 40 90

> sum(a)
[1] 60
> mean(a)
[1] 20
> length(a)
[1] 3
> sort(c(3,1,2))
[1] 1 2 3
```

Indexing Vectors

R indexing starts at 1, not 0. Elements are accessed with square brackets [].



`x <- c(12, 15, 18, 9, 21)` → `x[3]` selects the 3rd element

Positional `x[3]` Single element by position

Range `x[2:4]` A slice of elements

Multiple `x[c(1,3)]` Select several positions

Exclude `x[-1]` All except a position

Logical `x[x > 15]` Filter using a condition

Creating Matrices

A matrix is a 2-dimensional, rectangular arrangement of elements of a single data type — rows × columns.

1	3	5
2	4	6

```
matrix(1:6, nrow = 2, ncol = 3)
```

Filled column-wise by default (byrow = FALSE)

matrix()

Construct from a vector + dimensions

dim()

View or set dimensions

nrow() / ncol()

Count rows / columns

rbind() / cbind()

Combine by row / column

```
matrices.R

> m <- matrix(1:6, nrow = 2)
> m
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

> dim(m)
[1] 2 3

> m2 <- matrix(1:6, nrow=2,
               byrow = TRUE)

> rbind(c(1,2), c(3,4))
```


Matrix Operations

Element-wise

`+` `-` `*` `/`

Applied position by position between matrices of the same shape

Matrix Multiplication

`%*%`

True matrix product — inner dimensions must match

Transpose

`t(m)`

Flip rows and columns

Summaries

`rowSums()` `colSums()`
`rowMeans()`

Aggregate across rows or columns

```
matrix_ops.R

> A <- matrix(1:4, 2)
> B <- matrix(5:8, 2)

> A + B
> A %*% B

> t(A)

> rowSums(A)
[1] 4 6
> colMeans(A)
[1] 1.5 3.5
```

Indexing Matrices

Use `m[row, column]` — leave either side blank to select an entire row or column.

1	4	7
2	5	8
3	6	9

`m[2, ]` → entire 2nd row

`m[1, 2]`

Single element — row 1, column 2

`m[, 3]`

Entire 3rd column

`m[1:2, ]`

First two rows, all columns

`m[, c(1,3)]`

Columns 1 and 3

Creating Arrays

An array extends the matrix idea to more than two dimensions — think of it as stacked matrices ('slices').
`array(data, dim = c(rows, cols, slices))`



`dim = c(2, 3, 4)`

2 rows × 3 columns × 4 "slices"

- Same rules as matrices, one extra dimension per axis
- Every slice is a matrix of identical shape
- Common in biology for replicate × sample × time data

```
arrays.R

> arr <- array(1:24,
  dim = c(2, 3, 4))

> dim(arr)
[1] 2 3 4

> arr[, , 1]
      [,1] [,2] [,3]
[1,]     1     3     5
[2,]     2     4     6

> length(arr)
[1] 24
```

Indexing Array Elements

Extend matrix indexing with one bracket position per dimension: `arr[row, col, slice]`

```
arr[1, 2, 3]
```

Single element — row 1, col 2, slice 3

```
arr[, , 2]
```

Entire 2nd slice (a full matrix)

```
arr[1, , ]
```

1st row across every slice

```
arr[ , 1, ]
```

1st column across every slice

```
dim(arr) <- c(3, 2, 4)
```

Reshape an existing array

Arithmetic (+, −, *, /) on arrays is element-wise, exactly as with vectors and matrices — dimensions must match.

Key Takeaways

Vector

1-D · `c()`, `seq()` · `x[i]`, `x[cond]`

Matrix

2-D · `matrix()`, `rbind()/cbind()` · `m[r, c]`

Array

n-D · `array()`, `dim()` · `a[r, c, s]`

Remember: indexing in R starts at 1, operations are vectorised, and one data type per object (vector/matrix/array).

Thank You

